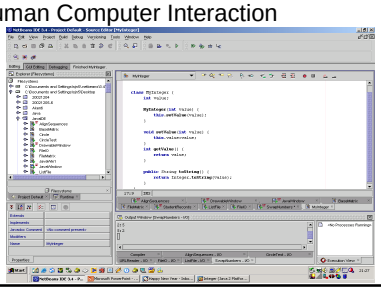
Graphics

Graphical User Interfaces

- Active area of research
- Human Computer Interaction



Java Swing

- A variety of classes to provide:
  - Menu bars, pull-down & pop-up menus
  - Windows & Frames
  - Radio & checkbox buttons
  - Tables & Trees
- Replaces Abstract Window Toolkit
  - Developed for initial Java release
  - Primary use now is in Applets
- Further information:
  - http://java.sun.com/docs/books/tutorial/uiswing/index.html

Model View Controller (MVC)

- Separates out data from presentation
  - Model: Underlying data content (Processing)
  - View: Presentation of the model (Output)
  - Controller: Mechanisms to change the model (Input)
- Very simple example…
  - What is the model?
  - What is the view?
  - What is the controller?

MVC

- Model
  - Boolean state
- View
  - Graphical representation
- Controller
  - Mouse click on button

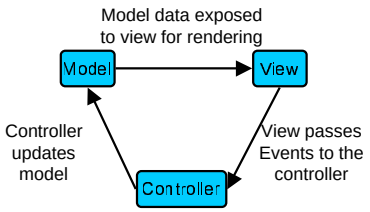
The CheckBox

```
import javax.swing.*;
public class JavaCheckButton {
  public static void main(String [] args) {
    JFrame window = new JFrame();
    window.setDefaultCloseOperation(
             JFrame.EXIT_ON_CLOSE);
    window.getContentPane().add(
             new JCheckBox("CheckBox"));
    window.pack();
    window.setVisible(true);
  }
}
```

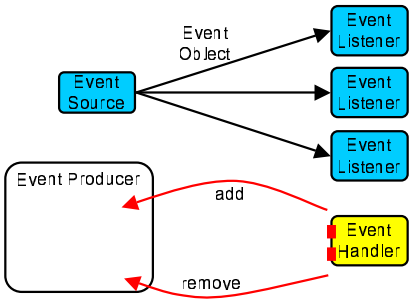

General Swing things…

- All ‘Swing Components’ have to appear within a top-level ‘Swing Container’:
  - **`JFrame`**: Main window with max, min & destroy icons
  - **`JDialog`**: Window dependent on a main window
  - **`JApplet`**: For applets only
- All Swing classes derived from **`JComponent`**
- All components are displayed within containers
- A container manages its own components
- A container may be added to another container

Events & MVC



Events: Publish & Subscribe



204

Expose Events

- Interaction with the GUI produces events
- To act on events need to add a listener
  - **`ActionListener:`** User clicks a button
  - **`WindowListener`**: User closes a frame (main window)
  - **`MouseListener`**: Mouse button pressed over a frame
  - **`MouseMotionListener`**: Mouse moves within a frame
  - **`ComponentListener`**: Component becomes visible
  - **`FocusListener`**: Component gets the keyboard focus
  - **`ListSelectionListener`**: Table or list selection changes
  - …

205

Expose the **`JCheckBox`** behaviour

- Main action of **`JCheckBox`**?
  - Toggle checkbox between true or false
- How do we 'intercept' this action?
  - Create a class that:
    - **`extend`**s a '**`Listener`**' class, or
    - **`implement`**s a '**`Listener`**' interface
  - Implement a method in the class to handle the event
  - Register an instance of this class with the component

206

Act on Events from the GUI

```
import javax.swing.*;
import java.awt.event.*;
public class JavaCheckButton {
  public static void main(String args[]) {
    JFrame window = new JFrame();
    JCheckBox checkBox = new JCheckBox("CheckBox");
    checkBox.addActionListener( new ActionListener () {
      public void actionPerformed(ActionEvent e) { {
        Object source = e.getSource();
        System.out.println( ((JCheckBox)source).isSelected());
      }
    });
    window.getContentPane().add(checkBox);
    window.pack();
    window.setVisible(true);
  }
}
```

```
true
false
true
false
```

207

Anonymous Inner Classes

```
……
    checkBox.addActionListener(
      new ActionListener() {
        public void actionPerformed(ActionEvent e) {
          …
        }
      }
    );
```

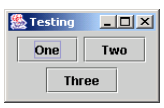
- Inherit the scope in which they were declared
- No need for a formal class declaration
- Within Swing they are used to:
  - Provide an implementation of an interface
  - Override the default behaviour of a standard class

Components within Containers

- **`JFrame`** is a top level Container
- **`JPanel`** is a generic Container
- Layout of Components within a Container is controlled by its **`LayoutManager`**
- Different **`LayoutManager`**s:
  - **`FlowLayout`**: Order left to right, top to bottom
  - **`GridLayout`**: Place components in a grid
  - **`BorderLayout`**: Place components in N,S,E,W & centre
  - ….

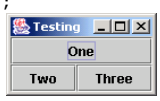
`FlowLayout` (default)

```
import javax.swing.*;
import java.awt.*;
class LayoutTest extends JPanel {
    public static void main(String [] args) {
        JFrame frame = new JFrame("Testing");
        frame.setLocation(200,200);
        frame.setContentPane(new LayoutTest());
        frame.pack();
        frame.setVisible(true);
    }
    LayoutTest() {
        add(new JButton("One"));
        add(new JButton("Two"));
        add(new JButton("Three"));
    }
}
```



`BorderLayout`

```
import javax.swing.*;
import java.awt.*;
class LayoutTest extends JPanel {
    public static void main(String [] args) {
        JFrame frame = new JFrame("Testing");
        frame.setLocation(200,200);
        frame.setContentPane(new LayoutTest());
        frame.pack();
        frame.setVisible(true);
    }
    LayoutTest() {
        setLayout(new BorderLayout());
        add(new JButton("One"),BorderLayout.NORTH);
        add(new JButton("Two"),BorderLayout.CENTER);
        add(new JButton("Three"),BorderLayout.EAST);
    }
}
```



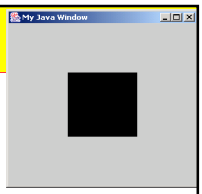
Redefining a **JComponent**

- The **paintComponent** method is called to redraw its screen representation

```
public void paintComponent(Graphics graphics) {
  super.paintComponent(graphics);
  // Redraw this component using the Graphics object
}
```

- The **repaint** method is used to request that the component is redrawn at some point in the future (through **paintComponent**)

MyWindow.java




```
import javax.swing.JFrame;
import javax.swing.JPanel;
import java.awt.Graphics;
import java.awt.Dimension;

public class MyWindow extends JFrame {
  private DrawableArea drawableArea;
  public MyWindow(String title) {
    super(title);
    drawableArea = new DrawableArea(new Dimension(300,300));
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    getContentPane().add(drawableArea);
    pack();
    setVisible(true);
  }
  public static void main(String[] args) {
    MyWindow window = new MyWindow("My Java Window");
    for (int i=0;i<50;i++) {
      window.moveObjects();
    }
  }
```


MyWindow.java (2/3)

```
public class DrawableArea extends JPanel {
  private ScreenObject object = new ScreenObject(50);
  class ScreenObject {
    int size,dX,dY;
    ScreenObject(int size) {
      this.size=size;
    }
    public void move() {
       dX+=1; dy+=1;
    }
    public void drawObject(Graphics graphics) {
      int posX = dX + getWidth()/2;
      int posY = dY + getHeight()/2;
      graphics.fillRect(posX-size,posY-size,size+size,size+size);
    }
  }

  public DrawableArea(Dimension size) {
    super(true);
    setPreferredSize(size);
  }
```

Inner class to represent the object within the **JPanel**

MyWindow.java (3/3)

```
    public void moveObjects() {
      object.move();
      repaint();
    }

    public void paintComponent(Graphics graphics) {
      super.paintComponent(graphics);
      object.drawObject(graphics);
    }
  }

  public void moveObjects() {
    drawableArea.moveObjects();
  }
}
```

If we had moved the **object** we need to ask that its screen representation be redrawn.